Batch Normalization

1. Batch Normalization (BatchNorm) is an **adaptive reparametrization technique** that stabilizes deep network training by normalizing layer inputs, reducing shifts in activation distributions over training.
2. It works by **standardizing activations** (zero mean, unit variance) within each mini-batch, then applying learned scale and shift parameters to maintain model expressiveness.
3. BatchNorm integrates normalization **within the model architecture**, allowing gradients to backpropagate through normalization operations, improving optimization behavior.

Batch Normalization

Batch Normalization solves the coordination problem in deep networks, where updating one layer changes the input distribution of subsequent layers. This phenomenon, often referred to as **internal covariate shift**, destabilizes and slows training.

In a deep network such as:

$$\hat{y} = xw_1w_2 \dots w_l$$

updating any weight w_i affects all subsequent layers. Higher-order interaction terms (e.g., second- and third-order Taylor contributions) make selecting a stable learning rate difficult as depth increases.

BatchNorm addresses this by **reparametrizing** the network:

- Normalizing activations within each mini-batch to **zero mean** and **unit variance**
- Introducing learnable parameters γ (scale) and β (shift) to preserve representational capacity

This reduces harmful interactions between layers and smooths the optimization landscape.

Short Need

To **stabilize and accelerate deep network training** by reducing sensitivity to parameter updates, smoothing the loss surface, and enabling higher learning rates.

Mathematical Formulation

Forward Pass

Given a mini-batch

$$\mathcal{B} = \{x_1, \ldots, x_m\}$$

with activations

$$H \in \mathbb{R}^{m \times d},$$

BatchNorm computes:

---

1. Mini-Batch Mean

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^{m} H_i \tag{8.36}$$

---

2. Mini-Batch Variance

$$\sigma_{\mathcal{B}}^2 = \delta + \frac{1}{m} \sum_{i=1}^{m} (H_i - \mu_{\mathcal{B}})^2 \tag{8.37}$$

where δ is a small constant (e.g., 10^{-8}) to avoid division by zero.

---

3. Normalize Activations

$$\hat{H}_i = \frac{H_i - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}}} \tag{8.35}$$

---

4. Scale and Shift

$$y_i = \gamma \hat{H}_i + \beta$$

where γ and β are learnable parameters.

Backward Pass

BatchNorm is fully differentiable:

- Gradients pass **through the mean and variance computations**.
- Normalization ensures gradients do not increase the mean or variance of activations, contributing to stable learning.

Inference Mode

During testing, mini-batches are not available.
BatchNorm uses the **running averages** collected during training:

$$y = \gamma \frac{x - \mathbb{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} + \beta$$

This allows deterministic evaluation on single examples.

Author's Message

BatchNorm is **not an optimization algorithm**; it is an **adaptive reparametrization technique** that makes optimization easier by:

- Reducing internal covariate shift
- Making deep network training more stable
- Avoiding destabilization from simultaneous layer updates
- Allowing higher learning rates
- Smoothing the optimization landscape

The key innovation is that normalization is part of the network itself and fully differentiable, enabling the optimizer to operate more effectively.